

Model-Free Prediction and Control

Learning policies without the environment model — Lecture 3/4

Etienne Chassaing

CentraleSupélec

1. Prediction Methods
2. Control Methods
3. Convergence & Analysis

By the end of this lecture, you should understand:

1. **Monte Carlo methods** — learn from complete trajectories
2. **Temporal Difference (TD) learning** — bootstrap and learn from partial trajectories
3. **Control algorithms** — SARSA (on-policy), Q-learning (off-policy)
4. **Bias-Variance trade-off** — MC vs TD vs Dynamic Programming
5. **Practical applications** — when to use each algorithm

Context: Lectures 1–2 assumed we knew the model. Here, we learn both prediction (estimate V) and control (find optimal π) by interacting only.

Goals of today's session

Complete this slide with session-specific learning objectives

Lecture 1–2: Planning

- Knew the model $p(s'|s, a)$
- Solved Bellman equations exactly
- Computed V_* or π_* in closed form

Two learning phases:

- **Prediction:** Given a policy π , learn its value function V^π
- **Control:** Learn to act optimally — find π_* through exploration

Lecture 3: Model-Free Learning

- Model unknown
- Learn from interaction (trial-and-error)
- Estimate V^π (prediction) or Q_* (control)
- Discover optimal policy by exploring

1. Prediction Methods

2. Control Methods

3. Convergence & Analysis

Idea: Average returns from complete episodes where we visit state s .

Algorithm:

1. Generate episodes following policy π
2. For each state visited, compute total return: $G_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T$
3. Update: $V(s) \leftarrow$ average of all observed G_t for that s

Properties:

- Unbiased: returns are true expected value
- High variance: depends on episode randomness
- Requires full episodes before updating

Temporal Difference (TD) Prediction

Idea: Bootstrap — use the Bellman equation as a learning rule:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Algorithm (TD(0)):

1. For each step: observe (s_t, r_{t+1}, s_{t+1})
2. Compute TD error: $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$
3. Update: $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$

Properties:

- Biased: uses current estimate $V(s_{t+1})$ as target
- Low variance: only one step ahead
- Updates during episodes (online learning)
- More sample efficient than MC

MC vs TD Comparison

	Monte Carlo	TD
Update	End of episode	Every step
Bias	None	Yes (bootstraps)
Variance	High	Low
Online	No	Yes
Sample efficiency	Lower	Higher
Convergence	Guaranteed	Guaranteed

Intuition: MC trades high variance for unbiasedness. TD trades bias for lower variance and online learning. Both converge to V^π under right conditions.

1. Prediction Methods

2. Control Methods

3. Convergence & Analysis

From Prediction to Control

Key insight: Use value estimates to improve the policy.

Policy Improvement: Given V^π , compute:

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Problem: We don't know $p(s' | s, a)$ in model-free setting!

Solution: Work with $Q(s, a)$ instead of $V(s)$:

$Q(s, a)$ = value of action a in state s

Then: $\pi(s) = \arg \max_a Q(s, a)$ — no model needed!

Two approaches:

- **On-Policy:** Learn Q^π while following π (SARSA)
- **Off-Policy:** Learn Q_* while following exploration policy (Q-learning)

On-Policy: SARSA

SARSA = State-Action-Reward-State-Action (the tuple we observe)

Algorithm:

1. Follow policy π (typically ϵ -greedy)
2. Observe tuple: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$
3. TD update for Q :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

4. Improve π : $\pi(s) = \epsilon$ -greedy($Q(s, \cdot)$)

Properties:

- On-policy: learns value of policy being followed
- Converges to Q^π (not necessarily optimal!)
- Conservative: exploration softens the bootstrap target

Off-Policy: Q-Learning

Q-Learning = Learn optimal Q_* while exploring

Algorithm:

1. Follow any exploration policy μ (e.g., ϵ -greedy)
2. Observe tuple: $(s_t, a_t, r_{t+1}, s_{t+1})$
3. TD update — always take max in target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

4. π is greedy w.r.t. current Q ; we explore with ϵ -greedy

Properties:

- Off-policy: learns Q_* despite exploring
- Converges to Q_* (optimal action-value)
- More aggressive: target is always the best action (max)
- Can reuse off-policy data

SARSA vs Q-Learning

	SARSA	Q-Learning
Policy	On-policy	Off-policy
Bootstrap target	$Q(s', a')$ (actual)	$\max_a Q(s', a)$ (best)
Converges to	Q^π	Q_*
Behavior	Conservative	Aggressive
Data reuse	No	Yes

Rule of thumb:

- **SARSA:** When you want to learn safely (respects the actual policy)
- **Q-Learning:** When you want the optimal policy (doesn't matter how you explore)

1. Prediction Methods
2. Control Methods
3. Convergence & Analysis

Convergence: MC vs TD vs DP

	DP	TD	MC
Requires model	Yes	No	No
Online	No	Yes	No
Sample efficient	N/A	High	Low
Bias	No	Yes	No
Variance	N/A	Low	High
Convergence	Guaranteed	Guaranteed	Guaranteed

Summary:

- **DP**: Perfect model, fast exact solution (Lecture 2)
- **TD**: No model, online, sample-efficient, biased (sweet spot)
- **MC**: No model, unbiased, high variance, episodic

Challenge: How to explore (find better policies) while exploiting (use best known action)?

ϵ -Greedy Strategy:

- With probability ϵ : take random action (explore)
- With probability $1 - \epsilon$: take best known action (exploit)
- Simple, effective, easy to implement
- Typically $\epsilon \approx 0.1$ (10% exploration)

Other exploration methods:

- Boltzmann: soft-max over values
- UCB: optimism under uncertainty
- Thompson sampling: posterior sampling

Key point: Without exploration, never find better policies. But too much exploration wastes time.

Key Takeaways

- **Prediction:** MC (unbiased, high var) vs TD (biased, low var, online)
- **Control:** SARSA (on-policy, conservative) vs Q-Learning (off-policy, optimal)
- **Trade-off spectrum:** DP $\rightarrow$ TD $\rightarrow$ MC in terms of bias/variance/sample-efficiency
- **Exploration:** Essential for learning; ϵ -greedy is simple baseline
- **Convergence:** All guaranteed under appropriate step sizes and exploration
- **Practical:** TD + ϵ -greedy often best balance for real problems

Questions?

Your feedback on this lecture is welcome

`etienne.chassaing@centralesupelec.fr`

 Survey form (anonymous feedback):

`forms.gle/your-form-link`

➡ Next: Lecture 4 — Advanced topics and applications!